02624 Inverse Problems and Imaging — Homework 3

Jeppe Klitgaard <s250250@dtu.dk>
Date: 2025-12-05

Contents

Introduction 0
1) 2D Heat Problem 1
  1.a) Ill-posedness of the 2D Backwards Heat Problem 1
  1.b) Implementation of the Forward Operator 1
  1.c) Application on Image 2
  1.d) Self-adjointness of the Forward Operator and Inversion using Landweber Iterations 2
    1.d.i) Self-adjointness of the Forward Operator 2
    1.d.ii) Inversion using Landweber Iterations 3
2) Total Variation Inpainting 4
  2.a) Displaying `KarinHarboStripe.png` 4
  2.b) TV and Harmonic Inpainting 5
  2.c) Inpainting Conclusions 6
A) Appendix 7
  A.1) Code preamble 7
  A.2) Code for Section 1.b forward operator 8
  A.3) Code for Section 1.b validation 9
  A.4) Code for Section 1.c validation 10
  A.5) Code for Section 1.d validation 11
  A.6) Code for Section 2.a validation 13
  A.7) Code for Section 2.b validation 13
  A.8) Code for Section 2.c validation 17

Introduction

This report presents solutions to Homework 3 for the course 02624 Inverse Problems and Imaging.

GitHub Copilot was used for tab completions in the development of the code associated with this report.

The code used to generate experiments and figures can be found in the appendix.

1) 2D Heat Problem

We consider the 2D backwards heat problem on the domain $(0, \pi)^2$ with homogenous Dirichlet boundary conditions as given by the forward operator

$$Kx(t_1, t_2) = \int_0^\pi \int_0^\pi k(t_1, t_2, s_1, s_2) x(s_1, s_2) \, \mathrm{d}s_1 \, \mathrm{d}s_2, \tag{1}$$

with kernel

$$k(t_1, t_2, s_1, s_2) = \sum_{n,m} e^{-(n^2+m^2)T} \sin(ns_1) \sin(ms_2) \sin(nt_1) \sin(mt_2) \tag{2}$$

where $x(s_1, s_2)$ is the image constituting the initial condition. The output of the model, $y(t_1, t_2) = Kx(t_1, t_2)$ represents the observed heat distribution at a time T.

1.a) Ill-posedness of the 2D Backwards Heat Problem

While the backwards heat problem may in some circumstances abide by both the existence and uniqueness criterion of the Hadamard definition of well-posedness, it does not lend itself to stable inversion, which can be appreciated by considering the exponential decay of the singular values with respect to time in the forward operator K. Upon inspection of (2), it is clear that the higher frequency components are rapidly attenuated as time T increases, which leads to instability with respect to even minute perturbations.

1.b) Implementation of the Forward Operator

Upon noticing that (1) constitutes a *convolution*, we realise that significant computational savings are available by leveraging the favourable scaling of the Fast Fourier Transform (FFT), by which we may transform to and from the frequency domain with $n \log(n)$ complexity. This allows us to carry out the convolution as pointwise multiplications in the frequency domain, which circumvents n^2 complexity entirely.

Given that we are working on a domain $(0, \pi)^2$ with homogenous Dirichlet boundary conditions, we choose to model only the interior of the domain, which also enables us to use the sine polynomial basis.

The implementation of the forward operator is given in appendix A.2.

We are able to validate our implementation by constructing a test case in which we only activate a single frequency component:

$$x_{\text{test}}(s_1, s_2) = \sin(ks_1) \sin(ls_2) \tag{3}$$

where we let $k = 4, l = 3$.

Inspection of the forward operator and (2) confirms that the inactive modes will remain zero, while the active mode will be attenuated by a factor

$$\alpha = e^{-(k^2+l^2)T}. \tag{4}$$

This allows us to compute an analytical solution y_{exact} and $y_{\text{numerical}}$, from which we can compute the error, e:

$$\begin{aligned} y_{\text{exact}}(t_1, t_2) &= \alpha \sin(kt_1) \sin(lt_2), \\ y_{\text{numerical}}(t_1, t_2) &= Kx_{\text{test}}(t_1, t_2), \\ e &= y_{\text{exact}} - y_{\text{numerical}}. \end{aligned} \tag{5}$$

The test case is visualised in Figure 1, which reveals that the error is of order $e \sim 10^{-20}$, which is negligible and may be attributed to numerical round-off errors.

As such, we conclude that our implementation of the forward operator is correct.

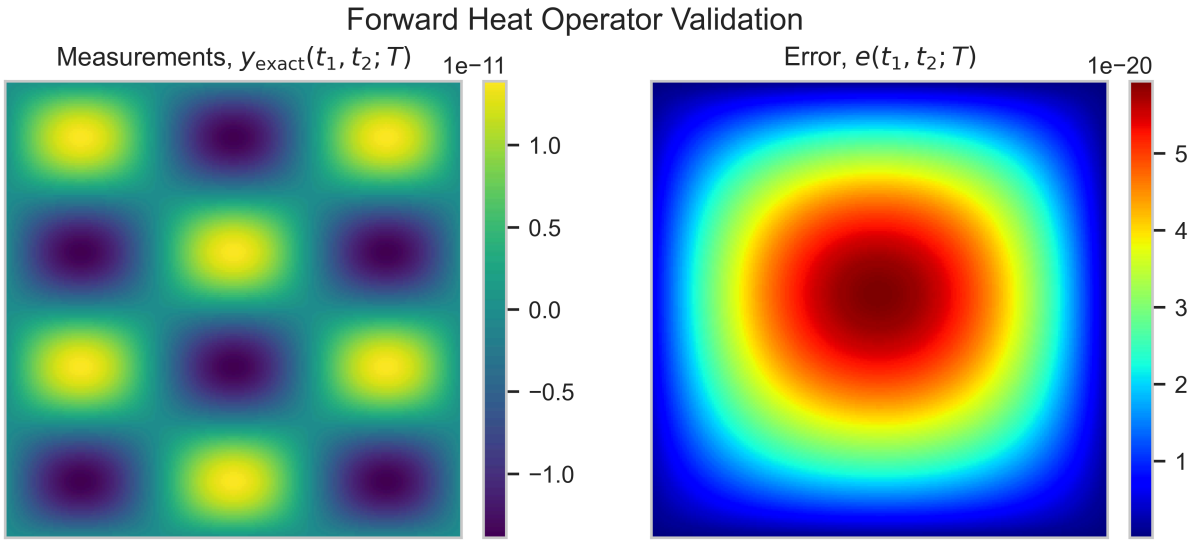


Figure 1: Test case for FFT-based implementation of the forward operator for the 2D heat equation.

1.c) Application on Image

We apply the forward operator on an image, `KarinHarbo512.png`, which is converted to a grayscale representation and normalised such that the pixel values lie in the interval $[0, 1]$.

This image is then passed through the forward operator with terminal time $T = 0.001$ and a Gaussian noise of standard deviation $\delta = 0.01$ is added to produce y^δ, which is depicted in Figure 2.

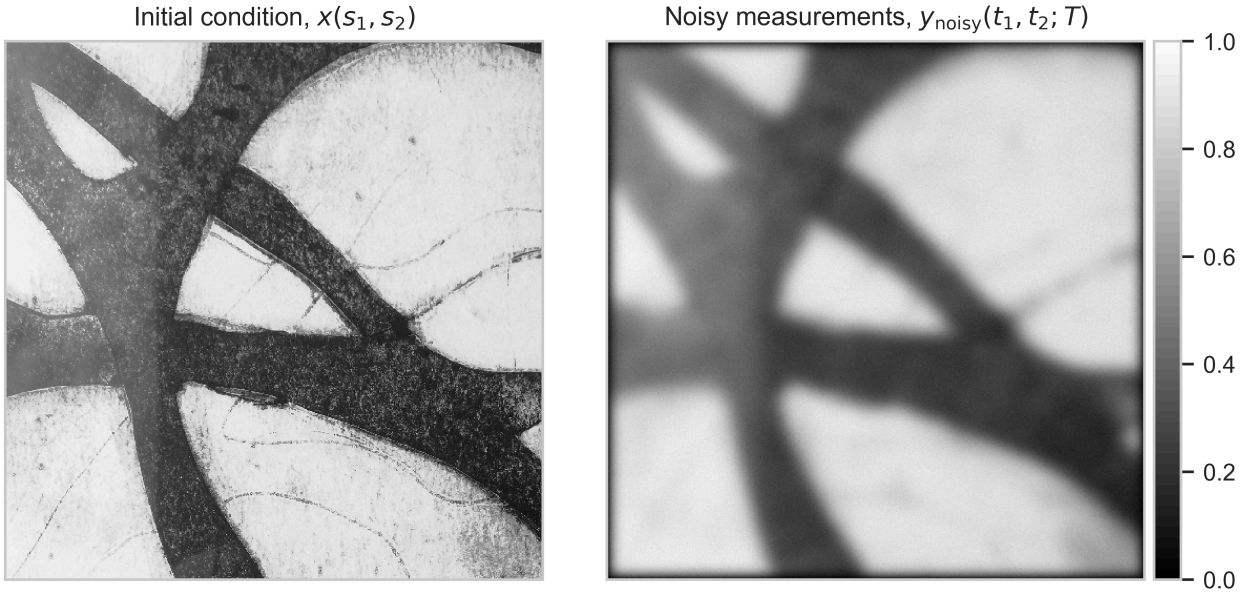


Figure 2: Application of the forward operator on image `KarinHarbo512.png` with added Gaussian noise.

1.d) Self-adjointness of the Forward Operator and Inversion using Landweber Iterations

1.d.i) Self-adjointness of the Forward Operator

Using a similar treatment as in Homework 1, we may prove self-adjointness of the forward operator K defined in (1) by considering its action on $x \in L^2((0, \pi)^2)$:

$$\begin{aligned} Kx(t_1, t_2) &= \int_0^\pi \int_0^\pi k(t_1, t_2, s_1, s_2) x(s_1, s_2) \, \mathrm{d}s_1 \, \mathrm{d}s_2 \\ &= \int_0^\pi \int_0^\pi k^*(\bar{s}_1, \bar{s}_2, t_1, t_2) x(s_1, s_2) \, \mathrm{d}s_1 \, \mathrm{d}s_2 = K^* x(t_1, t_2), \end{aligned} \tag{6}$$

Where k^* is the adjoint of the kernel k and $\bar{s}_i$ denotes the complex-conjugate of s_i. The second equality follows from symmetry of the kernel k:

$$
\begin{aligned}
k(t_1, t_2, s_1, s_2) &= \sum_{n,m} \beta_{nm} \sin(ns_1) \sin(ms_2) \sin(nt_1) \sin(mt_2) \\
&= k(s_1, s_2, t_1, t_2) = k^*(\bar{s}_1, \bar{s}_2, t_1, t_2)
\end{aligned} \tag{7}
$$

where $\beta_{nm} = e^{-(n^2+m^2)T}$ and $\bar{s}_i = s_i$ since $s_i \in \mathbb{R}$.

As such, from (6), we conclude that $K = K^*$ — that is, we conclude that the forward operator is self-adjoint.

An alternative approach would be to prove self-adjointness by equating the inner products $\langle Kx, y \rangle$ and $\langle x, Ky \rangle$ for arbitrary functions $x, y \in L^2((0, \pi)^2)$.

1.d.ii) Inversion using Landweber Iterations

This allows us to easily compute Landweber iterations, also referred to as *gradient descent* within optimisation theory, which we recall to be a semi-convergent iterative algorithm for solving ill-posed linear inverse problems. It may be expressed in the form

$$
x_{j+1} = x_j + \omega K^* \left(K x_j - y^\delta \right) \tag{8}
$$

Which given self-adjointness of K and approximation of K by a matrix A may be expressed as

$$
x_{j+1} = x_j + \omega A \left(A x_j - y^\delta \right) \tag{9}
$$

where j denotes the iteration index and ω is a relaxation parameter satisfying $0 < \omega < \frac{2}{\sigma_{\max}^2}$ with $\sigma_{\max}$ being the largest singular value of the operator $K \approx A$.

Given the sine polynomials as our basis, we find that the singular values are the eigenvalues of the operator, which may be read off directly from (2) as

$$
\sigma_{nm} = e^{-(n^2+m^2)T}. \tag{10}
$$

where $n, m \in \mathbb{N}_1$ implies that the largest singular value is given by $n = m = 1$, yielding

$$
\sigma_{\max} = e^{-2T}. \tag{11}
$$

We choose a relaxation parameter $\omega = \frac{9}{10} \cdot \frac{2}{\sigma_{\max}^2}$ where $\frac{9}{10}$ acts as a safety margin.

This lets us carry out $N_{\text{iter}} = 1500$ Landweber iterations, which produces Figure 3, a somewhat grainy reconstruction of the original image shown the left panel of Figure 2, though notably sharper than the blurred version shown in the right panel of the same figure.

We note that the reconstruction error is concentrated around the sharp edges of the original image, which is consistent with the analysis of Section 1.a, wherein we noted that higher frequency components are more severely attenuated by the forward operator, leading to more increased instability in their reconstruction.

Additionally, we observe a wave-like artefact around the borders of the image which we attribute to the fact that the original image does not tend to zero at the boundaries as required by the homogenous Dirichlet boundary conditions of the model. This continuity mismatch leads to the observed oscillations, which are reminiscent of the Gibbs phenomenon.

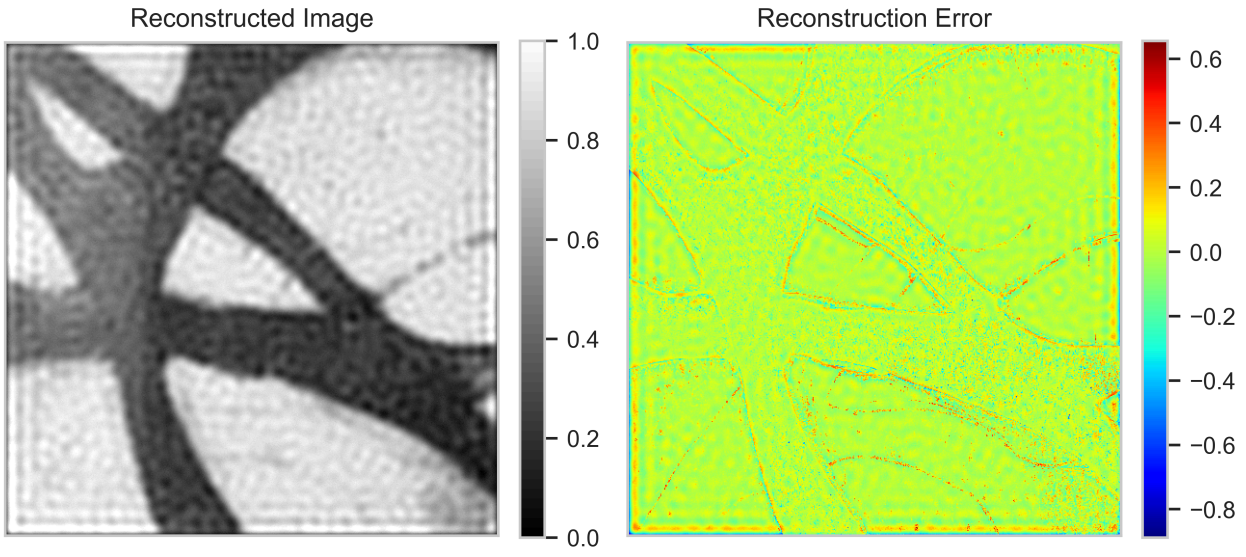


Figure 3: Landweber reconstruction of the initial condition from noisy observations at time $T = 0.001$ after $N_{\text{iter}} = 1500$ iterations.

Given the semi-convergence of Landweber iterations we choose to inspect the error history over the iterations calculated in the L^2 and L^∞-norms, which is depicted in Figure 4.

We can appreciate the form of Figure 4 by again considering the Landweber iterations as a gradient descent scheme, which makes it clear that it is semi-convergent in that it does not necessarily converge to the true solution, but rather to a local minimum of the residual norm, or in this case error norm. This is contrasted by the behaviour of other iterative methods such as the Conjugate Gradient (CG) method, in which the semi-convergence leads to 'overshooting' the minimum and divergence of the error norms after a certain number of iterations is observed. Lastly, we hypothesise that this could also happen in the case of Landweber iterations if instability is promoted through numerical errors or significant high-frequency noise.

This could be further investigated by increasing the number of allotted of iterations, N_{iter}, though at the cost of increased computation time and was thus not pursued further in this report.

Further iterations and alternative implementations using Conjugate Gradient (CG) may be investigated as part of the preparatory work for the examination.

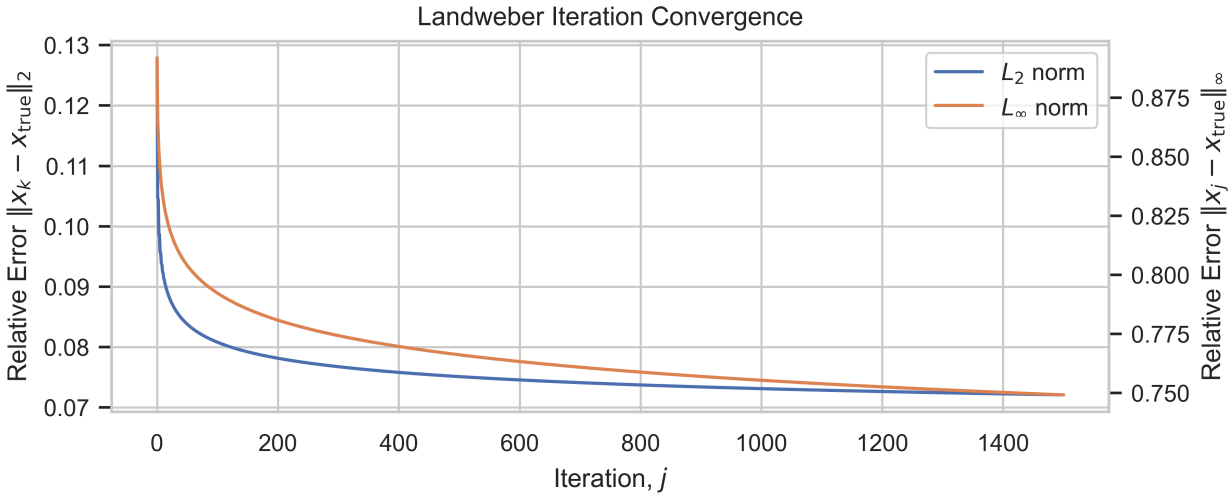


Figure 4: Error history of Landweber iterations in the L^2 and L^∞-norms.

2) Total Variation Inpainting

We recall the following variational formulation for total variation (TV) inpainting:

$$\min_u \frac{1}{2}\underbrace{\left\|\mathbb{1}_{\Omega\setminus\mathcal{M}}(z-u)\right\|^2}_{\text{data fidelity}} + \lambda\underbrace{\int_\Omega |\nabla u(x)|_2 \,\mathrm{d}x}_{\text{regularisation}} \qquad \text{TV Inpainting} \tag{12}$$

where λ is a regularisation parameter and z is the the remaining parts of the corrupted image as given by

$$z = \begin{cases} \bar{u} + \eta & \text{in } \Omega \setminus \mathcal{M} \\ \text{unknown} & \text{in } \mathcal{M} \end{cases} \tag{13}$$

with $\bar{u}$ being the original uncorrupted image, η being noise and $\mathcal{M}$ being the mask of corrupted pixels inside the full domain Ω.

We note that in the noiseless case, $\eta = 0$, the data fidelity term vanishes and the regularisation parameter, λ, may be set to unity without loss of generality.

We may also consider *harmonic* inpainting, which may be expressed on variational form as:

$$\min_{u\in H_0^1(\Omega)} \frac{1}{2}\left\|\mathbb{1}_{\Omega\setminus\mathcal{M}}(z-u)\right\|^2 + \frac{\lambda}{2}\|\nabla u(x)\|^2 \,\mathrm{d}x \qquad \text{Harmonic Inpainting} \tag{14}$$

Where $H_0^1(\Omega)$ denotes the Sobolev space of functions with square-integrable first derivatives that vanish on the boundary of Ω. and can be viewed as a generalised Tikhonov regularisation with H^1-norm penalty.

2.a) Displaying `KarinHarboStripe.png`

We display the partially corrupted image `KarinHarboStripe.png` by the same procedure used in Section 1.c, wherein the image is converted to grayscale and normalised to the interval $[0, 1]$ in order to produce Figure 5.

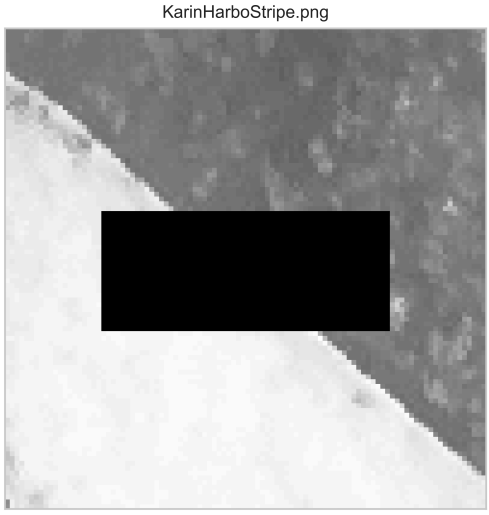


Figure 5: Display of the corrupted image `KarinHarboStripe.png`. The black region is the corrupted part, $\mathcal{M}$, to be inpainted.

2.b) TV and Harmonic Inpainting

We may solve the TV inpainting problem defined in (12) by formulating the problem in `cvxpy`, and qualitatively selecting $\lambda = 0.01$ as an optimal regularisation parameter which preserves most of the features of the uncorrupted image while inpainting the masked region.

With this, we are able to produce the reconstruction shown in the centre panel of Figure 6.

Alternatively, under the no-noise assumption, $\eta = 0$, we may set $\lambda = 1$ and solve the problem with the hard constraint $u = z \notin \mathcal{M}$ to produce the right panel of Figure 6.

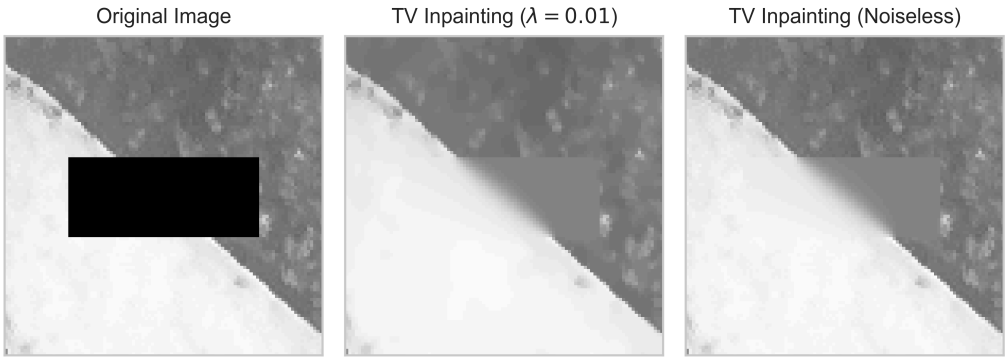


Figure 6: Total Variation inpainting given regularisation parameter $\lambda = 0.01$. The result under the noiseless assumption is shown in the right panel.

Similarly, we may solve the harmonic inpainting problem defined in (14) by again formulating the problem in `cvxpy`, and produce the reconstruction shown in the centre panel of Figure 7 with $\lambda = 0.1$, while the noiseless case is shown in the right panel of the same figure.

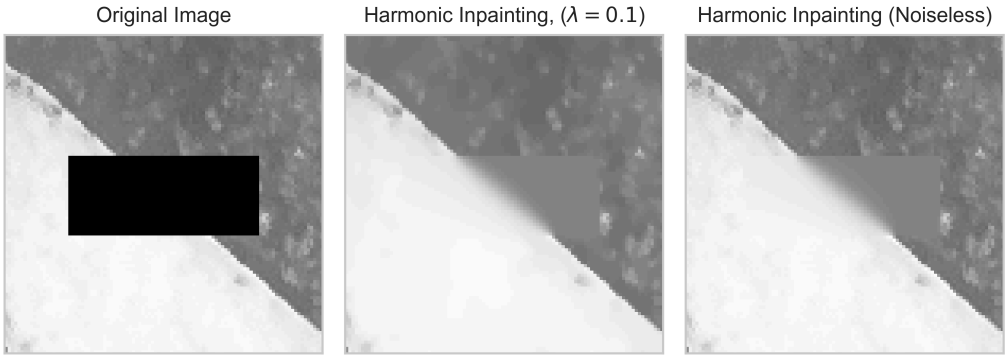


Figure 7: Harmonic inpainting given regularisation parameter $\lambda = 0.1$.

2.c) Inpainting Conclusions

We may collate the results from Section 2.b into a single figure for easier comparison, which is shown in Figure 8.

It is evident that the noiseless cases produce better results while having the additional benefit that correctly selecting the regularisation parameter is no longer required. However, this is only under the assumption that the image is truly noiseless to begin with, which is not likely to be the case.

In this case, the corrupted region contains a single, relatively straight line of high constrast, which lends itself to the TV inpainting method, which is able to reconstruct the line with high fidelity. However, it has the notable disadvantage that within both the light and dark regions of the image, an unnatural boxy artefact is introduced, which is smoothed out in the harmonic inpainting method. This can be understood by appreciating that the TV method seeks to reduce the total variation of the image, which promotes piecewise-constant solutions, while the harmonic method seeks to reduce the L^2-norm of the gradient, which promotes smooth solutions.

The harmonic method is not able to reconstruct the high-contrast line nearly as well as the TV method and instead lets the dark and light regions bleed into each other.

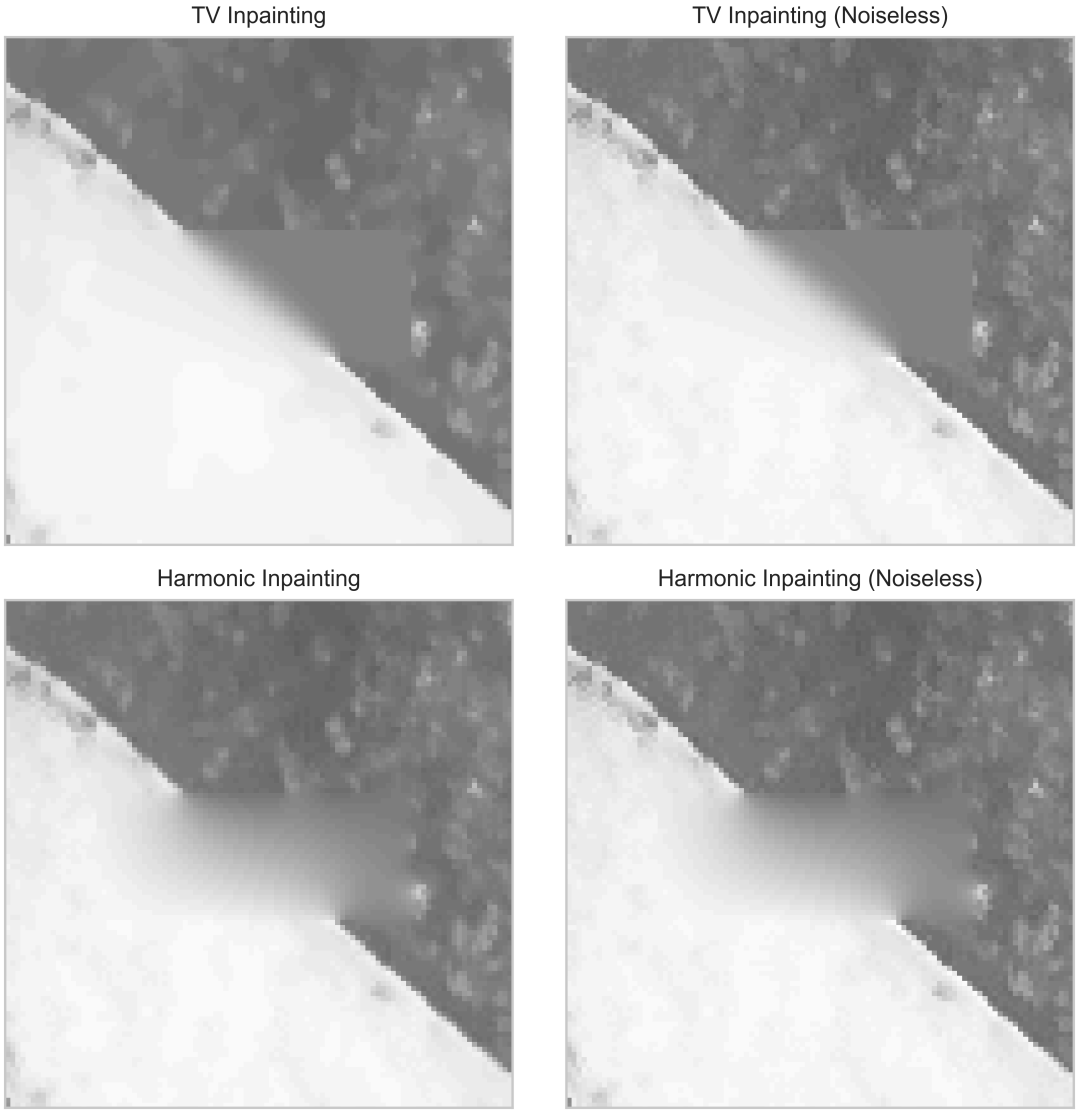


Figure 8: Comparison of inpainting methods.

A) Appendix

A.1) Code preamble

Python

```python
from pathlib import Path

import cvxpy as cp
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
import numpy.typing as npt
import pandas as pd
import seaborn as sns
from mpl_toolkits.axes_grid1 import make_axes_locatable

from PIL import Image as image
from scipy.fft import dst, idst
from tqdm.auto import tqdm, trange

# from numpy import fft
from pyfftw.interfaces import numpy_fft as fft
import pyfftw

# ---
HW3_DIR = Path().resolve()
OUTPUT_DIR = Path("./output")
OUTPUT_DIR.mkdir(exist_ok=True)

mm = 1 / 25.4  # mm in inches
PAPER_WIDTH_IN = (210 - 15 - 25) * mm

EXPORT_DPI = 600

sns.set_theme(context="paper", style="whitegrid")
gray_norm = mpl.colors.Normalize(vmin=0, vmax=1)

# ---
# Based on: https://stackoverflow.com/a/29261252/5036246
def sci_notation(number, sig_fig=2, latex_style=True, omit_coeff_if_unity=True):
    ret_string = "{0:.{1:d}e}".format(number, sig_fig)
    a, b = ret_string.split("e")

    # Calculate factor part
    factor = ""
    joiner = ""
    if omit_coeff_if_unity and float(a) == 1.0:
        factor = ""
        joiner = ""
    else:
        factor = a
        joiner = "×"

```

```
    # Calculate exponent part
    b = int(b)
    exponent = ""
    if latex_style:
        exponent = f"10^{{{str(b)}}}"
    else:
        exponent = f"10^{str(b)}"

    return factor + joiner + exponent

# ---
pyfftw.interfaces.cache.enable()
pyfftw.interfaces.cache.set_keepalive_time(60.0)  # A minute
```

A.2) Code for Section 1.b forward operator

```python
def forward_heat_operator(x: npt.NDArray, T: float) -> npt.NDArray:
    # An efficient way to solve this is to turn the convolution
    # into elementwise multiplication in the frequency domain
    # In Homework 2 we used DST, but here we are specifically asked to use FFT
    # So we follow odd-expansion as in Kim's given code (ModelProblems)

    N = x.shape[0]
    assert N == x.shape[1], "Input must be square"

    # Spatial scale
    ds = np.pi / (N + 1)

    # Wavenumbers
    N_ext = 2 * (N + 1)
    frequencies = fft.fftfreq(N_ext, d=ds)
    wavenumbers = 2 * np.pi * frequencies
    n = wavenumbers
    m = wavenumbers
    n_grid, m_grid = np.meshgrid(n, m, indexing="ij")

    # Decay factors
    k2 = n_grid**2 + m_grid**2  # Combined wavenumber
    decay_factors = np.exp(-(k2) * T)  # Here the same as eigvals or singvals

    # Construct odd expansion (Pad with zeros, reflect and negate)
    x_ext = np.zeros((N_ext, N_ext))  # Pad
    x_ext[1:N+1, 1:N+1] = x  # Insert original
    x_ext[N+2:, 1:N+1] = -np.flipud(x)  # Reflect along horizontal
    x_ext[:, N+2:] = -np.fliplr(x_ext[:, 1:N+1])  # Reflect along vertical

    # Carry out convolution in frequency domain
    x_ext_hat = fft.fft2(x_ext)
    x_ext_hat_T = x_ext_hat * decay_factors
    y_ext = fft.ifft2(x_ext_hat_T)

    y = y_ext[1:N+1, 1:N+1]
```

```
37
38      # Optionally, if we care about being normalised:
39      # Scaling factor due to basis functions sin(...) not being normalised
40      # scaling_factor = (np.pi / 2)**2
41      # y = y * scaling_factor
42      return np.real(y)
```

A.3) Code for Section 1.b validation

Python

```
1  # Construct test case from task
2  N = 256
3  k = 4
4  l = 3
5  T = 1
6
7  # Homogenous so don't store endpoints
8  ds = np.pi / (N + 1)
9  s_1 = np.arange(ds, np.pi, ds)
10 s_2 = s_1.copy()
11 s1_grid, s2_grid = np.meshgrid(s_1, s_2, indexing="ij")
12 x_initial = np.sin(k * s1_grid) * np.sin(l * s2_grid)
13 y_final = forward_heat_operator(x_initial, T=T)
14
15 # Test:
16 # Since only the k-lth mode is non-zero, only this mode is propagated
17 decay = np.exp(-(k**2 + l**2) * T)
18
19 # Optionally, if we care about being normalised:
20 # integral_scaling = (np.pi / 2)**2
21 # y_expected = x_initial * decay * integral_scaling
22 y_expected = x_initial * decay
23
24 plt.figure()
25 plt.imshow(x_initial)
26 plt.colorbar()
27 plt.title("Initial condition")
28
29 plt.figure()
30 plt.imshow(y_final)
31 plt.colorbar()
32 plt.title("Final condition (computed)")
33
34 plt.figure()
35 plt.imshow(y_expected)
36 plt.colorbar()
37 plt.title("Final condition (expected)")
38
39 plt.figure()
40 plt.imshow(y_final - y_expected)
41 plt.colorbar()
42 plt.title("Difference (computed - expected)")
43
```

```python
# ---
fig, (ax, ax_error) = plt.subplots(1, 2, figsize=(PAPER_WIDTH_IN, PAPER_WIDTH_IN / 2 - 0.5))

img = ax.imshow(y_expected, plt.cm.viridis)
ax.set_title("Measurements, $y_\\text{exact}(t_1, t_2; T)$")
ax.set_aspect('equal')
ax.set_xticks([])
ax.set_yticks([])

# https://stackoverflow.com/questions/18195758/set-matplotlib-colorbar-size-to-match-graph
plt.colorbar(img, ax=ax, fraction=0.046, pad=0.04)
img_error = ax_error.imshow(y_final - y_expected, plt.cm.jet)
ax_error.set_title("Error, $e(t_1, t_2; T)$")
ax_error.set_aspect('equal')
ax_error.set_xticks([])
ax_error.set_yticks([])
plt.colorbar(img_error, ax=ax_error, fraction=0.046, pad=0.04)

fig.tight_layout()
fig.suptitle("Forward Heat Operator Validation", y=1.02)

plt.savefig(OUTPUT_DIR / "1_b.png", dpi=EXPORT_DPI, bbox_inches='tight')
```

A.4) Code for Section 1.c validation

```python
img = image.open(HW3_DIR / "KarinHarbo512.png").convert("L")
x = np.asarray(img) / 255.0

T = 0.001
y = forward_heat_operator(x, T=T)

plt.figure()
plt.imshow(x, cmap='gray', norm=gray_norm)
plt.grid(False)
plt.figure()
plt.grid(False)
plt.imshow(y, cmap='gray', norm=gray_norm)

np.random.seed(42)
noise = np.random.normal(0, 0.01, size=y.shape)
y_noisy = y + noise
plt.figure()
plt.grid(False)
plt.imshow(y_noisy, cmap='gray', norm=gray_norm)

# plt.figure()
# plt.grid(False)
# plt.imshow(y - y_noisy, cmap='gray')

# ---
```

```python
fig, (ax_x, ax_y_noisy) = plt.subplots(1, 2, figsize=(PAPER_WIDTH_IN, PAPER_WIDTH_IN / 2 - 0.1), sharex=True)

img_x = ax_x.imshow(x, cmap='gray', norm=gray_norm)
img_y_noisy = ax_y_noisy.imshow(y_noisy, cmap='gray', norm=gray_norm)

for ax in (ax_x, ax_y_noisy):
    ax.grid(False)
    ax.set_xticks([])
    ax.set_yticks([])

# Add colorbars to both axes so they have equal size
# https://stackoverflow.com/questions/18195758/set-matplotlib-colorbar-size-to-match-graph
divider_x = make_axes_locatable(ax_x)
cax_x = divider_x.append_axes("right", size="5%", pad=0.05)
cax_x.axis('off')  # Hide the left colorbar axis

divider_y = make_axes_locatable(ax_y_noisy)
cax_y = divider_y.append_axes("right", size="5%", pad=0.05)
plt.colorbar(img_y_noisy, cax=cax_y)

ax_x.set_title("Initial condition, $x(s_1, s_2)$")
ax_y_noisy.set_title("Noisy measurements, $y_\\text{noisy}(t_1, t_2; T)$")

fig.tight_layout()
plt.savefig(OUTPUT_DIR / "1_c.png", dpi=EXPORT_DPI, bbox_inches='tight')
```

A.5) Code for Section 1.d validation

```python
# Since operator is self adjoint, we can easily implement Landweber

def landweber_iteration(x_k, y_noisy, T, omega):
    A_x_k = forward_heat_operator(x_k, T=T)
    return x_k - omega * forward_heat_operator(A_x_k - y_noisy, T=T)

# Largest singular value corresponds to the lowest frequency mode
# exp(-(k^2 + l^2) * T) with k=l=1
singular_value_max = np.exp(-2 * (1**2) * T)
omega = 0.9 * (2 / (singular_value_max**2))  # Safety margin
print(omega)
x_k = y_noisy.copy()
history = [x_k.copy()]
# N_iter = 1500
N_iter = 1500
for i in trange(N_iter):
    x_k = landweber_iteration(x_k, y_noisy, T, omega)
    history.append(x_k.copy())

# ---
fig, (ax_recon, ax_error) = plt.subplots(1, 2, figsize=(PAPER_WIDTH_IN, PAPER_WIDTH_IN / 2 - 0.1), sharex=True)
```

```
22
23 img_recon = ax_recon.imshow(history[-1], cmap='gray', norm=gray_norm)
24 img_error = ax_error.imshow(history[-1] - x, cmap='jet')
25 plt.colorbar(img_recon, ax=ax_recon, fraction=0.046, pad=0.04)
26 plt.colorbar(img_error, ax=ax_error, fraction=0.046, pad=0.04)
27
28 for ax in (ax_recon, ax_error):
29     ax.grid(False)
30     ax.set_xticks([])
31     ax.set_yticks([])
32
33 ax_recon.set_title("Reconstructed Image")
34 ax_error.set_title("Reconstruction Error")
35
36 fig.tight_layout()
37 plt.savefig(OUTPUT_DIR / "1_d_images.png", dpi=EXPORT_DPI, bbox_inches='tight')
38
39 # ---
40 data = []
41 for i, x_k in enumerate(tqdm(history)):
42     error_L2 = np.linalg.norm(x_k - x, ord=2) / np.linalg.norm(x, ord=2)
43     error_Linf = np.linalg.norm(x_k - x, ord=np.inf) / np.linalg.norm(x, ord=np.inf)
44     data.append({"Iteration": i, "Relative Error L2": error_L2, "Relative Error
       Linf": error_Linf})
45
46 df = pd.DataFrame(data)
47
48 # ---
49 plt.figure(figsize =(PAPER_WIDTH_IN, 2.5))
50
51 ax1 = plt.gca()
52 ax2 = ax1.twinx()
53
54 sns.lineplot(data=df, x="Iteration", y="Relative Error L2", label="$L_2$ norm",
   c="C0", ax=ax1)
55 sns.lineplot(data=df, x="Iteration", y="Relative Error Linf", label="$L_∞$ norm",
   c="C1", ax=ax2)
56
57 ax1.set_ylabel(r"Relative Error $\|x_k - x_\text{true}\|_2$")
58 ax2.set_ylabel(r"Relative Error $\|x_j - x_\text{true}\|_∞$")
59
60 # Collect labels in one legend
61 lines_1, labels_1 = ax1.get_legend_handles_labels()
62 lines_2, labels_2 = ax2.get_legend_handles_labels()
63 ax1.legend(lines_1 + lines_2, labels_1 + labels_2, loc='upper right')
64 ax2.legend_.remove()
65
66 ax2.grid(False)
67
68 plt.title("Landweber Iteration Convergence")
69 ax1.set_xlabel("Iteration, $j$")
```

```
70
71 plt.savefig(OUTPUT_DIR / "1_d_convergence.png", dpi=EXPORT_DPI, bbox_inches='tight')
```

A.6) Code for Section 2.a validation

Python

```
1  N = 100
2  img = image.open(HW3_DIR / "KarinHarboStripe.png").convert("L")
3  x = np.asarray(img) / 255.0
4
5  assert x.shape == (N, N)
6
7  # ---
8  plt.figure()
9  plt.imshow(x, cmap='gray', norm=gray_norm)
10 plt.xticks([])
11 plt.yticks([])
12 plt.title("KarinHarboStripe.png")
13
14 plt.savefig(OUTPUT_DIR / "2_a.png", dpi=EXPORT_DPI, bbox_inches='tight')
```

A.7) Code for Section 2.b validation

Python

```
1   # Generate mask
2   mask = x == 0.0
3
4   x_inpaint = x.copy()
5   # x_inpaint[mask] = 0.5  # Initial guess, start at gray
6   plt.figure()
7   plt.imshow(mask)
8   plt.figure()
9   plt.imshow(x_inpaint, cmap='gray', norm=gray_norm)
10
11  # x_
12
13  # ---
14  u = cp.Variable((N, N))
15
16  z = x_inpaint.copy()
17  M = (~mask).astype(np.float64)
18
19  fidelity = 0.5 * cp.sum_squares(cp.multiply(M, u - z))
20  regularisation = cp.tv(u)
21
22  lambda_ = 0.01
23  objective = cp.Minimize(fidelity + lambda_ * regularisation)
24  problem = cp.Problem(objective)
25  result = problem.solve()
26
27  u_sol_tv = u.value.copy()
28
29  # ---
30  plt.imshow(u_sol_tv, cmap='gray', norm=gray_norm)
31  plt.xticks([])
```

```
plt.yticks([])
plt.title("TV Inpainting Result")
plt.savefig(OUTPUT_DIR / "2_b_tv.png", dpi=EXPORT_DPI, bbox_inches='tight')


# ---
fig, (ax_orig, ax_inpaint) = plt.subplots(1, 2, figsize=(PAPER_WIDTH_IN, PAPER_WIDTH_IN / 2))

img = ax_orig.imshow(x_inpaint, cmap="gray", norm=gray_norm)
ax_orig.set_title("Original Image")
ax_orig.set_aspect('equal')
ax_orig.set_xticks([])
ax_orig.set_yticks([])

img_inpaint = ax_inpaint.imshow(u_sol_tv, cmap="gray", norm=gray_norm)
ax_inpaint.set_title("TV Inpainting")
ax_inpaint.set_aspect('equal')
ax_inpaint.set_xticks([])
ax_inpaint.set_yticks([])

fig.tight_layout()
plt.savefig(OUTPUT_DIR / "2_b_tv_noisy_comparison.png", dpi=EXPORT_DPI, bbox_inches='tight')

# ---
u = cp.Variable((N, N))
objective = cp.Minimize(cp.tv(u))
constraint = [u[~mask] == x_inpaint[~mask]]
problem = cp.Problem(objective, constraint)
result = problem.solve()

u_sol_tv_noiseless = u.value.copy()

# ---
fig, (ax_orig, ax_inpaint) = plt.subplots(1, 2, figsize=(PAPER_WIDTH_IN, PAPER_WIDTH_IN / 2))

img = ax_orig.imshow(x_inpaint, cmap="gray", norm=gray_norm)
ax_orig.set_title("Original Image")
ax_orig.set_aspect('equal')
ax_orig.set_xticks([])
ax_orig.set_yticks([])

img_inpaint = ax_inpaint.imshow(u_sol_tv_noiseless, cmap="gray", norm=gray_norm)
ax_inpaint.set_title("TV Inpainting (Noiseless)")
ax_inpaint.set_aspect('equal')
ax_inpaint.set_xticks([])
ax_inpaint.set_yticks([])

fig.tight_layout()
```

```
plt.savefig(OUTPUT_DIR / "2_b_tv_noiseless_comparison.png", dpi=EXPORT_DPI, bbox_inches='tight')

# ---
fig, axs = plt.subplots(1, 3, figsize=(PAPER_WIDTH_IN, PAPER_WIDTH_IN / 2))

img = ax_orig.imshow(x_inpaint, cmap="gray", norm=gray_norm)
axs[0].imshow(x_inpaint, cmap="gray", norm=gray_norm)
axs[0].set_title("Original Image")

axs[1].imshow(u_sol_tv, cmap="gray", norm=gray_norm)
axs[1].set_title("TV Inpainting ($λ=0.01$)")

axs[2].imshow(u_sol_tv_noiseless, cmap="gray", norm=gray_norm)
axs[2].set_title("TV Inpainting (Noiseless)")

for ax in axs:
    ax.set_aspect('equal')
    ax.set_xticks([])
    ax.set_yticks([])

fig.tight_layout()
plt.savefig(OUTPUT_DIR / "2_b_tv_comparison.png", dpi=EXPORT_DPI, bbox_inches='tight')

# ---
# Harmonic
u = cp.Variable((N, N))

z = x_inpaint.copy()
M = (~mask).astype(np.float64)
fidelity = 0.5 * cp.sum_squares(cp.multiply(M, u - z))

grad_u_x = cp.diff(u, axis=0)
grad_u_y = cp.diff(u, axis=1)
regularisation =  0.5 * (cp.sum_squares(grad_u_x) + cp.sum_squares(grad_u_y))

lambda_ = 0.1
objective = cp.Minimize(fidelity + lambda_ * regularisation)
problem = cp.Problem(objective)
result = problem.solve()

u_sol_harmonic = u.value.copy()

# ---
plt.figure(figsize = (60 * mm, 60 * mm))
plt.imshow(u_sol_harmonic, cmap='gray')
plt.xticks([])
plt.yticks([])
plt.title("Harmonic Inpainting Result")
plt.savefig(OUTPUT_DIR / "2_b_harmonic.png", dpi=EXPORT_DPI, bbox_inches='tight')
```

```
129
130 # ---
131 fig, (ax_orig, ax_inpaint) = plt.subplots(1, 2, figsize=(PAPER_WIDTH_IN,
    PAPER_WIDTH_IN / 2))
132
133 img = ax_orig.imshow(x_inpaint, cmap="gray", norm=gray_norm)
134 ax_orig.set_title("Original Image")
135 ax_orig.set_aspect('equal')
136 ax_orig.set_xticks([])
137 ax_orig.set_yticks([])
138
139 img_inpaint = ax_inpaint.imshow(u_sol_harmonic, cmap="gray", norm=gray_norm)
140 ax_inpaint.set_title("Harmonic Inpainting")
141 ax_inpaint.set_aspect('equal')
142 ax_inpaint.set_xticks([])
143 ax_inpaint.set_yticks([])
144
145 fig.tight_layout()
146 plt.savefig(OUTPUT_DIR / "2_b_harmonic_noisy_comparison.png", dpi=EXPORT_DPI,
    bbox_inches='tight')
147
148 # ---
149 u = cp.Variable((N, N))
150 grad_u_x = cp.diff(u, axis=0)
151 grad_u_y = cp.diff(u, axis=1)
152 objective = cp.Minimize(cp.sum_squares(grad_u_x) + cp.sum_squares(grad_u_y))
153 constraint = [u[~mask] == x_inpaint[~mask]]
154 problem = cp.Problem(objective, constraint)
155 result = problem.solve()
156
157 u_sol_harmonic_noiseless = u.value.copy()
158
159 # ---
160 fig, (ax_orig, ax_inpaint) = plt.subplots(1, 2, figsize=(PAPER_WIDTH_IN,
    PAPER_WIDTH_IN / 2))
161
162 img = ax_orig.imshow(x_inpaint, cmap="gray", norm=gray_norm)
163 ax_orig.set_title("Original Image")
164 ax_orig.set_aspect('equal')
165 ax_orig.set_xticks([])
166 ax_orig.set_yticks([])
167
168 img_inpaint = ax_inpaint.imshow(u_sol_harmonic_noiseless, cmap="gray",
    norm=gray_norm)
169 ax_inpaint.set_title("Harmonic Inpainting (Noiseless)")
170 ax_inpaint.set_aspect('equal')
171 ax_inpaint.set_xticks([])
172 ax_inpaint.set_yticks([])
173
174 fig.tight_layout()
175 plt.savefig(OUTPUT_DIR / "2_b_harmonic_noiseless_comparison.png", dpi=EXPORT_DPI,
    bbox_inches='tight')
```

```

# ---
fig, axs = plt.subplots(1, 3, figsize=(PAPER_WIDTH_IN, PAPER_WIDTH_IN / 2))

img = ax_orig.imshow(x_inpaint, cmap="gray", norm=gray_norm)
axs[0].imshow(x_inpaint, cmap="gray", norm=gray_norm)
axs[0].set_title("Original Image")

axs[1].imshow(u_sol_tv, cmap="gray", norm=gray_norm)
axs[1].set_title("Harmonic Inpainting, ($λ=0.1$)")

axs[2].imshow(u_sol_tv_noiseless, cmap="gray", norm=gray_norm)
axs[2].set_title("Harmonic Inpainting (Noiseless)")

for ax in axs:
    ax.set_aspect('equal')
    ax.set_xticks([])
    ax.set_yticks([])

fig.tight_layout()
plt.savefig(OUTPUT_DIR / "2_b_harmonic_comparison.png", dpi=EXPORT_DPI,
bbox_inches='tight')
```

A.8) Code for Section 2.c validation

```python
fig, axs = plt.subplots(2, 2, figsize=(PAPER_WIDTH_IN, PAPER_WIDTH_IN))

axs[0, 0].imshow(u_sol_tv, cmap="gray", norm=gray_norm)
axs[0, 0].set_title("TV Inpainting")

axs[0, 1].imshow(u_sol_tv_noiseless, cmap="gray", norm=gray_norm)
axs[0, 1].set_title("TV Inpainting (Noiseless)")

axs[1, 0].imshow(u_sol_harmonic, cmap="gray", norm=gray_norm)
axs[1, 0].set_title("Harmonic Inpainting")

axs[1, 1].imshow(u_sol_harmonic_noiseless, cmap="gray", norm=gray_norm)
axs[1, 1].set_title("Harmonic Inpainting (Noiseless)")

for ax in axs.flatten():
    ax.set_aspect('equal')
    ax.set_xticks([])
    ax.set_yticks([])

fig.tight_layout()
fig.savefig(OUTPUT_DIR / "2_c.png", dpi=EXPORT_DPI, bbox_inches='tight')
```